



Türkiye'nin sanatçı odaklı merch pazaryeri

Operator Runbooks

Step-by-step procedures for operational tasks

Audience · Operations, DevOps, on-call rotation

Date · 2026-05-28

Source · docs/RUNBOOKS.md

Contents

1. Backup Restore Drill
2. KVKK Data Subject Request (Erasure / Access / Portability)
3. Soft-Launch Plan
4. Google Search Console + Bing Webmaster Submission
5. Email Deliverability Health Check
6. Production Incident Response
7. Vendor Onboarding Smoke Test (5 min check)
8. Pre-Deploy Checklist (when shipping changes)

1. Backup Restore Drill

Goal: Prove the daily Postgres backup is actually restorable. Measure RTO (recovery time objective).

Frequency: Run this drill at least once before launch, then quarterly.

Prerequisites:

- SSH access to VPS (`ssh root@vibehub.com.tr`)
- Local Docker installed
- ~30 minutes of focused time
- A non-production directory to test in (e.g., `~/vibehub-restore-test/`)

Procedure:

```
# 1. SSH to VPS and find the latest backup
ssh root@vibehub.com.tr
ls -lh /root/vibehub/backups/ | tail -5
# Pick the most recent .sql.gz file, note its size

# 2. Copy it to your local machine
exit
scp root@vibehub.com.tr:/root/vibehub/backups/<latest>.sql.gz ~/vibehub-restore-test/

# 3. Spin up a fresh Postgres locally
docker run -d --name restore-test \
  -e POSTGRES_USER=test \
  -e POSTGRES_PASSWORD=test \
  -e POSTGRES_DB=vibehub_restored \
  -p 15432:5432 \
  postgres:16-alpine

# 4. Wait for it to be ready
sleep 5
docker exec restore-test pg_isready -U test

# 5. Restore the backup — START THE CLOCK ⌚
time gunzip -c ~/vibehub-restore-test/<latest>.sql.gz | \
  docker exec -i restore-test psql -U test -d vibehub_restored

# 6. Sanity checks — does the data look right?
docker exec restore-test psql -U test -d vibehub_restored -c \
  "SELECT COUNT(*) FROM \"User\";"
docker exec restore-test psql -U test -d vibehub_restored -c \
  "SELECT COUNT(*), MAX(\"createdAt\") FROM \"Order\";"
docker exec restore-test psql -U test -d vibehub_restored -c \
  "SELECT slug, status FROM \"Tenant\";"

# 7. (Optional) Point a local backend at the restored DB to prove the app works
# Override DATABASE_URL=postgresql://test:test@localhost:15432/vibehub_restored
# Run `cd backend && npm run start:dev`
# Hit /auth/me or /admin/overview — should respond with restored data

# 8. Cleanup
docker stop restore-test && docker rm restore-test
rm ~/vibehub-restore-test/<latest>.sql.gz
```

What to record:

- ⌚ Restore time = your RTO
- Backup file size
- Last `Order.createdAt` (your RPO is the gap between this and "now")
- Any errors during restore

Acceptance criteria:

- Restore completes without errors

- Row counts match production within 24h margin
- Last order timestamp is within 24h of the backup time

Red flags:

- Restore time > 30 min → consider streaming replication
 - pg_dump errors during restore → backup script may be broken
 - Missing tables → pg_dump was filtering schemas; review backup command
-

2. KVKK Data Subject Request (Erasure / Access / Portability)

Goal: Handle a user's KVKK Article 11 request to access, export, or delete their personal data.

Legal context: Turkish KVKK gives users:

- Right to know what data is held about them (Access)
- Right to receive that data in a portable format (Portability)
- Right to have their data erased (Erasure / "Right to be forgotten")
- Right to correct inaccurate data (Rectification)

SLA: Must respond within 30 days per KVKK regulation.

2a. Access Request

User asks: "What personal data do you have on me?"

Procedure:

```
# SSH to VPS
ssh root@vibehub.com.tr
cd /root/vibehub

# Run a one-off Prisma query (write to a temp file)
cat > /tmp/kvkk-access.js <<'EOF'
const { PrismaClient } = require('@prisma/client');
const p = new PrismaClient();
const EMAIL = process.argv[2];
(async () => {
  const user = await p.user.findUnique({
    where: { email: EMAIL },
    include: {
      profile: true,
      orders: { include: { items: true } },
      reviews: true,
      wishlist: true,
      trustedDevices: { select: { id: true, label: true, lastSeenAt: true, createdAt: true } },
      pushDevices: { select: { id: true, platform: true, lastSeenAt: true } },
      messagesSent: { select: { id: true, recipientId: true, body: true, createdAt: true } },
      messagesReceived: { select: { id: true, senderId: true, body: true, createdAt: true } },
      notifications: true,
      forumTopics: true,
      forumReplies: true,
      profileVisitsMade: true,
      profileVisitsReceived: true,
      follows: true,
      auditLogsAs: { select: { id: true, action: true, targetType: true, createdAt: true } },
    },
  });
  if (!user) { console.error('No user found'); process.exit(1); }
  console.log(JSON.stringify(user, null, 2));
  await p.$disconnect();
})();
EOF
docker compose exec -T backend node /tmp/kvkk-access.js USER_EMAIL > /tmp/kvkk-export-USER_EMAIL.json
```

Send the JSON file to the user via secure channel (NOT email). Use a password-protected ZIP if possible.

2b. Erasure Request

User asks: "Delete my account and all my data."

Procedure:

1. **Verify identity** — request from the email on file, OR a phone call with details only they would know.

2. Check for blockers:

- Are there open orders (status `PLACED`, `CONFIRMED`, `SHIPPED`)? If yes, ask user to wait until delivered/cancelled, OR cancel them admin-side first.
- Are there pending refunds? Process them first.

3. Execute the deletion via the app:

- User logs in → `/profile/settings` → Delete account
- This calls `DELETE /auth/account` which is now KVKK-compliant per `auth.service.ts` (anonymizes Orders/Reviews/Forum, purges PII, keeps AuditLog with `actorId=null` for forensics)

4. If user can't log in:

- Admin can trigger deletion server-side. SSH to VPS:

```
docker compose exec backend node -e "  
const { PrismaClient } = require('@prisma/client');  
const p = new PrismaClient();  
(async () => {  
  const u = await p.user.findUnique({ where: { email: 'USER_EMAIL' } });  
  if (!u) { console.error('not found'); return; }  
  // Call the same service the API uses (DON'T hand-craft a delete - FK  
    constraints)  
  // For now, log into the app as that user via admin password reset → then  
    trigger.  
})();  
"
```

⚠ Don't try to write the deletion SQL by hand. The service handles ~15 tables and FK ordering correctly.

5. **Confirmation email** — Send the user written confirmation that data was deleted, including the date. Keep a copy in your records (NOT linked to the user — anonymized).

2c. Portability Request

Same as Access (above), but you must provide the data in a machine-readable format (JSON or CSV). The JSON from the Access script satisfies this.

3. Soft-Launch Plan

Goal: Stress-test the platform with friendly users before opening to the public.

Current state: 4 GOD_USERS in DB, 4 test tenants (kalt, modexl, tekir + 1). Zero real customers, zero real orders.

Phase 1: Internal Dogfood (1 week)

Participants: You + 1-2 trusted devs/friends.

Goal: Catch the obvious "the button doesn't work" bugs.

Tasks:

- Each person creates an account, browses, adds items to cart, places a mock order
- Each person tests the refund flow end-to-end (request → admin approves → return barcode → mark arrived → refund completes)
- One person plays "vendor": apply, get approved (you self-approve), upload products, get a "sale", mark shipped, view payouts
- Track every bug in a spreadsheet — assign priority + owner

Phase 2: Closed Beta (2 weeks)

Participants: 5-10 friendly users — friends, family, fellow musicians.

Vendor recruitment: 3-5 real artists who agree to test the vendor flow. Pay them a small honorarium for their time (50 TL gift card?). They onboard, add 5-10 products each.

Tasks:

- Beta users get an invite code or signup link
- Real lyzico **sandbox** mode (NOT production — no real money)
- Daily check-in: any blocker? Any confusion?
- Weekly retrospective: what would prevent them from recommending VibeHub?

Success criteria:

- All beta users complete at least one full purchase flow
- All vendors complete at least one sale + fulfillment
- Zero PO bugs
- Vendor activation rate (time from approval → first sale) < 7 days

Phase 3: Public Launch (after Phase 2 closes clean)

Pre-launch checklist:

☐

All PO bugs from beta resolved

☐

lyzico production keys obtained + tested

☐

Real kargo provider integration live (or manual fulfillment SOP documented)

☐

Backup restore drill completed (RTO measured)

☐

Sentry alerts configured (queue depth, error rate)

☐

Google Search Console set up (see runbook 4)

☐

Press release drafted (if doing PR)

☐

Customer support email tested (`support@vibehub.com.tr` → reaches you within 15 min)

☐

KVKK Aydınlatma Metni reviewed by a lawyer

☐

Privacy policy updated to reflect actual data flows

☐

Maintenance mode toggle tested

Launch day:

- Toggle vendor signups OPEN
- Email beta users that public launch is live (they get +1 month free shipping?)
- Monitor Sentry dashboard continuously for first 6 hours
- Have a rollback plan: `git revert HEAD && git push` → auto-deploys

4. Google Search Console + Bing Webmaster Submission

Goal: Get the site indexed by Google and Bing as fast as possible.

Prerequisites:

- Domain `vibehub.com.tr` is live
- `https://vibehub.com.tr/google-site-verification.html` is reachable (already in `public/`)
- Google account
- Microsoft account (for Bing)

4a. Google Search Console

1. Go to <https://search.google.com/search-console/>
2. Click **Add Property** → **URL prefix** → `https://vibehub.com.tr`
3. Verification method: **HTML file** → it'll show a filename like `google1234567890abcdef.html`
4. **Important:** The file you have is `public/google-site-verification.html` — check what filename Google gives you. If different, rename or copy yours to match.
5. Click **Verify**.
6. Once verified, in the left sidebar:
 - **Sitemaps** → add `https://vibehub.com.tr/sitemap.xml` → Submit

- **URL Inspection** → paste `https://vibehub.com.tr/` → **Request Indexing** (manual nudge)
- Repeat URL inspection for top 5 important pages (`/` , `/shop` , `/about` , `/faq` , `/legal`)

7. **Enhancements** tab (will populate over 24-72h):

- Check **Mobile Usability**, **Core Web Vitals**, **Page Experience**
- Fix any errors that show up

4b. Bing Webmaster Tools

1. Go to <https://www.bing.com/webmasters/>
2. **Import from Google Search Console** (one-click — easiest path)
3. If preferring manual: same procedure as Google, but the verification meta tag or XML file goes to `public/`
4. Add sitemap URL

4c. Validate Rich Results

After submitting:

1. Go to <https://search.google.com/test/rich-results>
2. Paste a product URL (`https://vibehub.com.tr/product/<real-id>`)
3. Should show: Product schema (with offer/aggregateRating), BreadcrumbList. Fix any errors.
4. Repeat for `/store/<slug>` , `/faq` , home

4d. Monitor Weekly (first month)

- Check **Coverage** report — any "Excluded" or "Errored" pages?
- Check **Performance** — clicks, impressions, average position. Should grow over 4-8 weeks.
- Submit new high-value pages via URL Inspection if not auto-indexed within 7 days.

5. Email Deliverability Health Check

Goal: Verify transactional emails (OTP, order confirm, refund) actually reach inboxes — not spam folders.

Frequency: Once before launch, monthly after.

Procedure:

1. **Test with major providers** — sign up for VibeHub with throwaway accounts on:
 - Gmail

- Outlook / Hotmail
- Yahoo
- iCloud
- A Turkish provider (Yandex / Mail.com.tr)

2. For each:

- Register → does OTP email arrive within 30 sec?
- Check inbox vs spam folder
- View email source: SPF/DKIM/DMARC pass?

3. Use mail-tester.com:

- Trigger an email to the random address mail-tester gives you
- Score should be $\geq 9/10$. Below that, fix what it flags.

4. DNS checks:

```
dig TXT vibehub.com.tr | grep -i spf
dig TXT _dmarc.vibehub.com.tr
dig TXT default._domainkey.vibehub.com.tr # or whatever selector
```

Red flags:

- Mail-tester score < 8 → SPF/DKIM not configured properly
- Gmail → spam folder → likely DMARC issue or low domain reputation
- Yahoo bouncing → bounce handling not implemented in BullMQ

6. Production Incident Response

Goal: When something is on fire, follow these steps.

6a. Site is down (5xx errors)

```
# 1. SSH to VPS
ssh root@vibehub.com.tr
cd /root/vibehub

# 2. Check container status
docker compose ps

# 3. Check recent logs (last 200 lines)
docker compose logs --tail=200 backend
docker compose logs --tail=200 frontend

# 4. If a container is restarting in a loop, check it specifically
docker compose logs --tail=500 -f backend

# 5. Restart the offending container
docker compose restart backend
# or full restart:
docker compose down && docker compose up -d

# 6. If still broken, rollback to last known good commit
git log --oneline -10
git reset --hard <previous-good-sha>
docker compose up -d --build

# 7. Once stable, debug the issue locally and write a fix
```

6b. Database performance issue

```
# Check active queries
docker compose exec postgres psql -U vibehub -c "
SELECT pid, now() - query_start AS duration, state, query
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY duration DESC LIMIT 20;"

# Kill a stuck query (after deciding it should be killed)
docker compose exec postgres psql -U vibehub -c "SELECT pg_cancel_backend(<pid>);"

# Check table sizes
docker compose exec postgres psql -U vibehub -c "
SELECT relname, pg_size_pretty(pg_total_relation_size(relid))
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC LIMIT 10;"
```

6c. Email queue backed up

```
# Check Redis queue depth via the new admin endpoint
curl -H "Authorization: Bearer <admin-token>" \
  https://api.vibehub.com.tr/admin/queue-health

# If queue is huge, check Redis directly
docker compose exec redis redis-cli -a "$REDIS_PASSWORD" KEYS "bull:mail:*" | wc -l

# Clear failed jobs (last resort — investigates first why they failed)
docker compose exec backend node -e "
const Queue = require('bullmq').Queue;
const q = new Queue('mail', { connection: { host: 'redis', port: 6379, password:
  process.env.REDIS_PASSWORD } });
q.clean(0, 1000, 'failed').then(removed => console.log('Removed:', removed));
"
```

6d. Security event (suspicious activity)

```
# Check audit log for the last hour, critical events only
docker compose exec postgres psql -U vibehub -c "
SELECT \"createdAt\", action, \"actorId\", \"targetType\", \"targetId\", metadata
FROM \"AuditLog\"
WHERE \"createdAt\" > NOW() - INTERVAL '1 hour'
AND action IN ('ACCOUNT_LOCKED', 'ADMIN_USER_PASSWORD_RESET',
  'PLATFORM_SETTINGS_UPDATE', 'TRAP_ROUTE_HIT', 'LOGIN_FAILED')
ORDER BY \"createdAt\" DESC LIMIT 50;"

# If you see attack patterns, enable IP-level blocking via Nginx
# (specifics depend on your Nginx config)
```

7. Vendor Onboarding Smoke Test (5 min check)

When: Every Monday, OR after deploying changes to vendor flow.

Procedure:

1. Open <https://vibehub.com.tr/vendors/apply> in incognito
2. Submit a test application with `vendor-test+$(date +%s)@vibehub.com.tr`
3. Login as admin → `/dashboard/admin/vendors` → "Pending" tab → approve
4. Login as the new vendor → `/dashboard/vendor` → does the dashboard render?
5. Try to add a product → does it save?
6. Try to upload an image → does it appear?
7. Add a variant → save → does the product list show stock?
8. Clean up: delete the test vendor account from admin panel

Pass criteria: All steps complete in < 5 minutes, zero 403/500 errors.

Fail criteria: Any 403 = permission grant problem. Any 500 = file an incident.

8. Pre-Deploy Checklist (when shipping changes)

Before pushing to `main`:

☐

`cd backend && npx tsc --noEmit` exits 0

☐

`cd frontend && npx tsc --noEmit` exits 0

☐

`cd frontend && npm run build` exits 0

☐

Manual spot-check on at least one page that was changed

☐

If schema changed: `npx prisma migrate dev --name <descriptive>` ran locally

☐

If env vars changed: updated `.env.example` AND `docs/INFRASTRUCTURE.md`

☐

Commit message follows repo style (see `git log --oneline -10`)

☐

No `.env` or secrets accidentally staged: `git diff --cached | grep -E "(KEY|SECRET|PASS|TOKEN)="` returns nothing

After push:

- Watch GitHub Actions → must go green
- Watch VPS deploy logs:
`ssh root@vibehub.com.tr 'docker compose logs --tail=50 backend'`
- Smoke-check the changed feature on prod within 10 min